Java Introduction (Detailed Guide for PDF)

What is Java?

Java is a high-level, object-oriented, and platform-independent programming language used to build applications.

It follows the principle: "Write Once, Run Anywhere" (WORA).

History of Java

- Developed by James Gosling at Sun Microsystems
- Released in 1995
- Now owned by Oracle

Why Use Java?

- Platform independent
- Secure and robust
- Widely used in enterprise applications
- Strong community support

Features of Java

- Object-Oriented
- Platform Independent
- Secure
- Multithreaded
- Robust

Java Program Structure

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
```

```
    }
}
```

Variables

```
int x = 10;
String name = "John";
```

Data Types

- int
- float
- double
- char
- boolean

Operators

- Arithmetic (+, -, *, /)
- Relational (==, !=)
- Logical (&&, ||)

Conditional Statements

```
int age = 20;

if (age > 18) {
    System.out.println("Adult");
} else {
    System.out.println("Minor");
}
```

Loops

For Loop

```
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

While Loop

```
int i = 0;
while (i < 5) {
    i++;
}
```

Methods (Functions)

```
public static void greet() {
    System.out.println("Hello");
}
```

Object-Oriented Programming

Class and Object

```
class Person {
    String name;
}
```

Inheritance

Polymorphism

Encapsulation

Arrays

```
int[] numbers = {1, 2, 3};
```

Exception Handling

```
try {
    int x = 10 / 0;
} catch (Exception e) {
    System.out.println("Error");
}
```

Java Packages

Used to organize classes.

```
import java.util.*;
```

Example Project

```
import java.util.Scanner;

public class Test {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String name = sc.nextLine();
        System.out.println("Hello " + name);
    }
}
```

Advantages of Java

- Platform independent
- Secure
- Scalable

Disadvantages of Java

- Slower than some languages
- Requires more memory

Best Practices

- Follow naming conventions
- Use proper indentation
- Write modular code